

No AI JS Heap

Implemented with a binary tree based heap.

Basic rules that define what makes a binary tree a heap and will inform how to construct and manage the heap.

A binary tree is considered a **heap** if it satisfies two specific conditions:

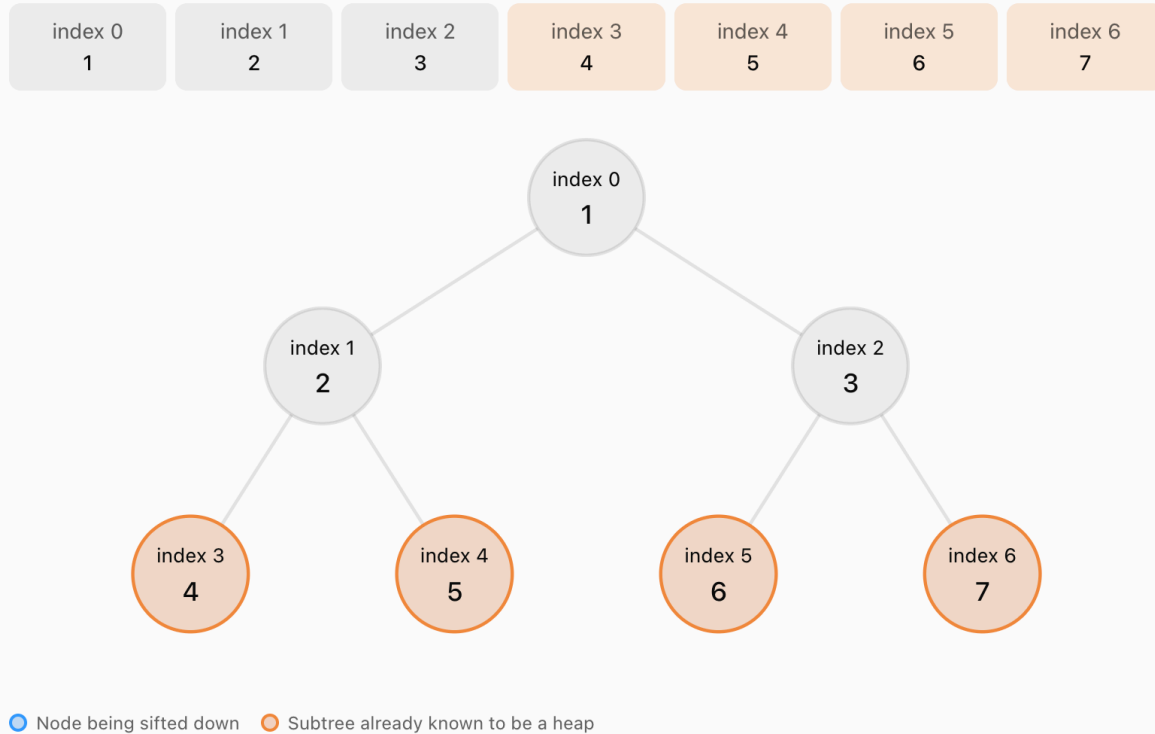
- **Shape Property (Completeness):** The tree must be a **complete binary tree**. This means every level, except possibly the last, is completely filled, and all nodes on the last level are as far left as possible.
- **Heap Property (Ordering):** Each node's value must be either $\geq$ (for a max heap) or $\leq$ (min heap) than it's children.

Keep in mind that a binary tree heap does not have an ordering requirement between the left and right children. Normally you may expect that the binary tree would have the smallest value always to the left or right and the largest value consistently on the other side. But this is not true for a binary tree based heap. Only the **Heap Property** must be maintained.

Constructor and Pop

Understanding the constructor and pop loop was confusing for me at first. I wanted to understand why the middle of the array (technically just before the middle) was chosen to start the loop where we start calling siftDown from. This is because all nodes from before the start of the middle of the array to the beginning are guaranteed to be leaf nodes as seen below:

Why heap construction moves from the last parent to the root



Start: indexes 3–6 are leaves. Each leaf is already a valid one-node heap, so no work is needed below index 2.

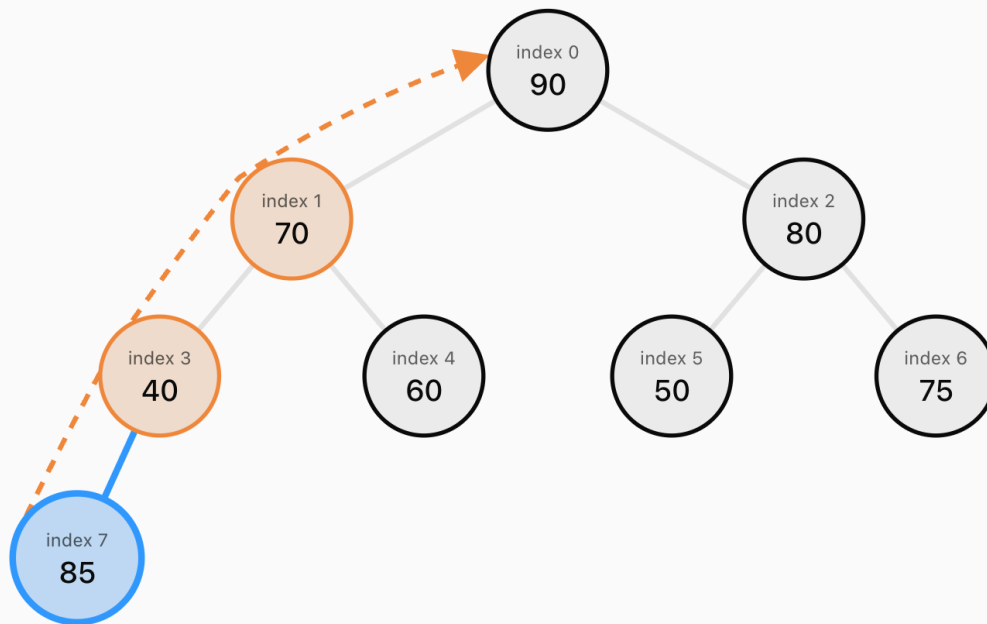
siftDown will work from the node just before the leaves, working it's way back to the root of the tree replacing a node with it's child node if that node is larger (in the case of a max heap) or smaller (for min heap).

siftDown works by taking the nodes with leaves and swapping their values

Insert

For we start by adding a node to the end of the array backing the heap. Then with insert we need to siftUp starting from the end of the array moving back up the tree ancestor by ancestor until we know we have moved the new leaf node, swapping, until the newly inserted leaf node reaches it's place in the sub trees it is a part of.

Max heap after append — before siftUp



$85 > 40$ — the new leaf must swap with its parent first.

● Newly inserted leaf ● Possible ancestor path

Array representation

0	1	2	3	4	5	6	7
90	70	80	40	60	50	75	85